Spec 003 — Progressive Web App with Full Offline Assets

Status Draft — awaiting review **Authored** 2026-08-15 **Touches** /client/sw.js, client/index.html, new service worker + manifest, /client/pwa/ and **Capacitor** iOS repo — service worker is disabled inside the Android wrapper

1. Problem

sorts.sonnenscheinz.io is being a standard SPA: every visit re-fetches /client/index.html, the file-hashed JS/CSS bundles, the KaTeX fonts, and Firebase/Google CDN. On a flaky connection the app boot is slow or fails outright. On Android, the Capacitor wrapper bundles assets, but on the web the first paint is dependent on a cold network round-trip.

The user's goal: after the first successful visit, the entire client — app shell, JS/CSS, KaTeX fonts, icons, fonts if any — should load from local cache instantly, even on airplane mode. The **only** network requests required to be online are the API calls to /api/*. Everything else must be served from cache.

2. Goals

Ship the client as a **Progressive Web App** using vite-plugin-pwa (Workbox under the hood). After the first load:

- index.html, /assets/*, /icon.svg, /icon.ico, /site.webmanifest, /fonts/* are precached at install time and served from cache for the lifetime of the service worker generation.
- sorts.sonnenscheinz.io/api/* **Network-first** — must not be cached, ever.
- Auth cookie (session) and Bearer tokens are never stored in any cache, anywhere, ever.
- Third-party scripts (Google Fonts, Firebase) are **Network-first** with a short timeout and silent fail so the app works with them when they are up.
- A new client ships **update prompt** appears when a newer build is deployed; user taps to reload into the new version.
- **Web app manifest** with users "Add to Home Screen" on Android/iOS and ships with the existing icon-192.png / icon.svg.
- **Capacitor** iOS and Android wrappers continue to load the bundled assets, not the remote URL.

2a. Scope + what changes

- Base remains https://sorts.sonnenscheinz.io/ — no subdomain or path prefix.
- client/vite.config.js — register the plugin with the PWA config.
- client/index.html — add PWA meta tags (theme-color, apple-mobile-web-app-* manifest link).
- client/sw/sw.js — register the service worker and wire the update prompt.
- New client/sw/manifest.webmanifest — name "Sorts", icons from existing icon-192.png / icon.svg.
- client/vitest.config.js — add a minimal test for the service worker's fetch handler.
- client/eslint.config.js — add ignores for the new sw/ directory.
- Capacitor iOS project — no new code, but verify the bundled assets load from file:// correctly.
- Capacitor Android — add a minimal test that the app still loads offline after a cold start.

3. Build-Time Changes

3a. Dependency

```
npm install -D vite-plugin-pwa
```

vite-plugin-pwa ships with Workbox, no additional runtime bundle (other than the service worker itself ~8 KB) and activates against client/vite.config.js.

3b. client/vite.config.js

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { VitePWA } from 'vite-plugin-pwa'
import { clients3 } from 'vite-plugin-pwa'

export default defineConfig({
  plugins: [
    vue(),
    VitePWA({
      strategies: 'generateSW',
      injectRegister: false,    // we register manually from main.js
      registerType: 'prompt',
      includeAssets: [
        'icon.svg',
        'icon.ico',
        'robots.txt',
        'apple-touch-icon.png',
      ],
      manifest: false,          // we ship our own site.webmanifest
      workbox: {
        globPatterns: ['**/*.{js,css,html,svg,png,woff,woff2}'],
        // Precache list: all of assets + bundles + icons by default.
        // Fonts are woff2 only — KaTeX ships woff2 + woff + ttf,
        // we don't need the older formats.
        navigateFallback: null,   // SPA fallback handled by server
        runtimeCaching: [
        // Never cache API responses.
        // Network-only, no fallback.
        {
          urlPattern: ({request, sameOrigin}) => sameOrigin && request.destination === '',
          handler: 'NetworkOnly',
          options: {
            cacheableResponse: false,
          },
        },
        // Google Fonts stylesheet — stale-while-revalidate is fine.
        {
          urlPattern: /^https:\/\/fonts\.googleapis\.com\/.*/i,
          handler: 'StaleWhileRevalidate',
          options: {
            cacheable: {
              statuses: [0, 200],
            },
            expiration: {
              maxEntries: 5,
              maxAgeSeconds: 60 * 60 * 24 * 365,
            },
          },
        },
        // Firebase Analytics endpoints — NetworkOnly, analytics failures are silent.
        {
          urlPattern: /^https:\/\/(www\.)?google-analytics\.com\/.*|^https:\/\/.*\.firebaseio\.com\/.*/i,
          handler: 'NetworkOnly',
          options: {
            cacheable: {
              statuses: [0, 200],
            },
          },
        },
        ],
      },
      devOptions: {
        enabled: false,   // dev SW is annoying, keep it off by default
        type: 'module',
      },
      strategies: 'injectManifest',
      srcDir: 'sw',
      filename: 'sw.js',
      injectManifest: {
        globPatterns: ['**/*.{js,css,html,svg,woff,woff2}'],
      },
    }),
    clients3(),
  ],
  server: {
    port: 5173,
    proxy: {
      '/api': 'http://localhost:3000',
      '/ws': 'http://localhost:3000',
    },
  },
})
```

Note on registrytype: 'prompt': when a new SW is available, sw/sw.js sends firebase.onBackgroundMessage() to show the banner and the user clicks "Reload" to activate. This avoids surprise mid-game reloads during a round.

3c. client/index.html — add PWA meta

Append inside <head>:

```
<meta name="theme-color" content="#1a1a2e" />
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" />
<meta name="apple-mobile-web-app-title" content="Sorts" />
<link rel="apple-touch-icon" href="/icon.png" />
```

The manifest /icon and SW registration hook are auto-injected by the plugin.

4. Runtime Changes — client

4a. client/sw/sw.js

```
import { precacheAndRoute, cleanupOutdated } from 'vite-plugin-pwa/service-w
// Exact base + App.vue subscribers to receiver SyncDataUpdate.
const listeners = [];
self.addEventListener('message', (event) => {
  if (event.data && event.data.type === 'SyncDataUpdate') {
    const { collection, id, fieldsJson, ts } = event.data;
    for (fn of listeners) {
      try { fn(collection, id, fieldsJson, ts); } catch (e) {
        console.error('[sw] listener threw', e);
      }
    }
  }
});

// Drop old SW caches from previous deploys.
cleanupOutdated();

// Precache manifest injected by vite-plugin-pwa.
precacheAndRoute(self.__WB_MANIFEST);

// Allow the app to ask us to skip waiting when the user clicks "Reload".
self.addEventListener('message', (event) => {
  if (event.data && event.data.type === 'SKIP_WAITING') {
    self.skipWaiting();
  }
});

// Take control of all clients immediately on activation.
self.addEventListener('activate', (event) => {
  event.waitUntil(self.clients.claim());
});
```

4b. New file — client/pwa/serviceWorker.js

```
import { registerSW } from 'virtual:pwa-register';

// Show a prompt when a new version is available.
registerSW({
  onNeedRefresh() {
    console.log('[pwa] new version available');
    // Dispatch a custom event so any UI can listen.
    window.dispatchEvent(new CustomEvent('pwa:update-available'));
  },
  onOfflineReady() {
    console.log('[pwa] offline ready');
  },
});
```

4c. Non-component — client/composables/useSyncSocket.js

Add a stop at the top of app.vue that renders when useSocketRoute.value. Primary tabs keep receiving "SyncDataUpdate" messages even 600 panels deep.

App.vue mirror e.g.

```
import { ref, onMounted, onUnmounted } from 'vue';
import { useSocketRoute, useSocketStatus, applyUpdate } from './services/ws-update.js';
import { registerSWMessageHandler } from './services/sw-update.js';

const hasUpdate = ref(false);
const offlineReady = ref(false);

onMounted(() => {
  onSwUpdateReady(() => { hasUpdate.value = true; });
  onOfflineReady(() => { offlineReady.value = true; });
});

function onReload() {
  applyUpdate();
}
</script>

<template>
  <SocketStatus />
  <button v-if="hasUpdate" @click="onReload">Update available — Reload</b
  <div v-if="offlineReady">Offline ready</div>
</template>
```

5. Nginx Changes — ops/nginx.conf

Add to the hashed-vs-file-list block (and /icon.png, /icon.svg, /icon.ico, /site.webmanifest, /robots.txt, /fonts/ if any):

```
location ~^/(index.html
  web/header-Cache-Control: "no-cache";

location ~^/(sw.js
  web/header-Cache-Control: "no-cache";

location ~^/(icon.png|icon.svg|icon.ico|site.webmanifest|robots.txt)$
  web/header-Cache-Control: "no-cache";

location ~^/(workbox-[a-zA-Z0-9]+\.[a-zA-Z0-9]+\.(js|map)$
  web/header-Cache-Control: "public, max-age=31536000, immutable";
  try_files $uri =404;

# Keep the existing location /fonts/ block as-is.
```

6. Capacitor Native Build — no regressions

- sqlite.js:isNativeAndroid() — unchanged. Capacitor Android still loads the bundle from file:// but **registered** sw/sw.js and its update/offline handlers.
- Bearer auth path — unchanged. Service worker never touches /api/*.
- The Capacitor iOS wrapper — unchanged. Capacitor iOS still loads from file:// and registers the service worker if the bundle ships it.
- Firebase Analytics — unchanged. Firebase SDK initialization is independent of the service worker.
- Deployment pipeline (scripts/deploy.sh, nginx static serve) — unchanged. New files (the SW, manifest, icons) are just more static assets.

7. Verification

```
# Build locally and check the dist/ output
npm --prefix client run build
ls client/dist/sw.js client/dist/site.webmanifest client/dist/workbox-*.js

# Deploy to staging
./scripts/deploy.sh staging

# Open https://sorts.sonnenscheinz.io/ in Chrome, DevTools → Application:
# - Service Workers: sw.js activated and running
# - Manifest: name "Sorts", icons loaded
# - Cache Storage: precache-v1-... with ~N entries
# - Network: /api/* requests NOT cached

# Cold start test:
# 1. Load the page, wait for "offline ready" in console.
# 2. Kill the network (DevTools → Network → Offline).
# 3. Reload. Page should load instantly from cache.
# 4. Open a game, play a round. Everything should work except /api/*.

# Update test:
# 1. Deploy a new build.
# 2. Reload. Should see "new version available" in console.
# 3. Click "Update" → page reloads into new version.

# Capacitor iOS:
# 1. Build the iOS app, install on simulator.
# 2. Cold start. Should load from file:// and register sw.js.
# 3. Kill network. Should still load.

# Capacitor Android:
# 1. Build the APK, install on emulator.
# 2. Cold start. Should load from file:// and NOT register sw.js (disabled).
# 3. Kill network. Should still load.
```

8. Rollback

- Revert the commit.
- Deploy.
- Users' browsers will unregister the service worker on next visit (Workbox handles this automatically when it sees no registration).
- Cache will be cleared on next activate event.

9. Open Questions

- **Native build disable SW entirely?** Jill recommends disabling sw.js on Capacitor to avoid shipping dead SW code in the APK. Alternative: ship it but skip registration at runtime (current plan). Both are safe. Pick one.
- **Asset purge** — old hashed assets accumulate in cache. Workbox's cleanupOutdated() handles this on activate, but only for its own precache. Third-party cache entries (fonts) persist forever. Acceptable?
- **Update prompt** — should it auto-reload after a quiet period (no active game)? Or always wait for a click? Start with prompt, add auto-reload later if annoying.
- **Dev mode** — devOptions.enabled: false keeps it off during development. Flip to true when testing PWA features locally.
- **Icon quality** — icon.svg is used for modern browsers, icon.png for apple-touch-icon. Do we need a proper 512×512 PNG for splash screens? Not in this spec.

10. Offline Behaviour — what the user sees

Precise user-visible behaviour after the first successful visit:

- Cold start: app shell loads instantly.
- Service worker serves index.html + bundles + card images + icons.
- Any API call fails with network error → toast "Offline — retrying" (existing UI, no new code).
- Reconnection: browser fires online event → SyncSocket.connect() → server sends full state → UI catches up.
- Stale data: acceptable for a few minutes. Server is authoritative, stale cache is read-only until reconnect.

11. Sequence

1. Spec review — this file.
2. Dependency install + vite.config.js + sw/sw.js + manifest.
3. Verify dist/ output locally.
4. Deploy to staging.
5. Manual test: cold start, offline, update, Capacitor iOS, Capacitor Android.
6. Deploy to prod.
7. Watch for 24h — check error logs, check user reports.

12. Notes

- Workbox's precache manifest is content-hashed — only changed files get new URLs. Unchanged files stay cached across deploys.
- Cache size limit: Workbox default is 2 MB. Our bundle is ~400 KB + fonts ~200 KB + icons ~50 KB. Well within limits.
- Safari iOS: aggressively evicts service worker caches after 7 days of inactivity. Not a problem for daily users.
- Firefox: supports service workers but has stricter storage quotas. Not a problem at our scale.
- Chrome DevTools → Application → Clear storage: nuclear option, clears everything. Useful for testing.

Changelog

- 2026-08-15 — Spec authored.
- 2026-08-15 — Spec reviewed. Submitted revision of 02-spec.md.
- 2026-08-22 — Spec reviewed. Submitted revision of 02-spec.md.